ChatGPT

Overview: PyTorch PrivateUse1 and Custom Device Backends

PyTorch provides *reserved dispatch keys* (`PrivateUse1`, `PrivateUse2`, `PrivateUse3` and corresponding autograd keys) to enable **out-of-tree hardware integration** without modifying PyTorch core [1]. Rather than requesting a new dispatch key (which are limited in number and require upstream changes), hardware vendors can use `PrivateUse1` to implement their device support externally. This approach became *recommended* as more accelerators emerged, to avoid constantly adding keys in core PyTorch [2]. Early on, the PrivateUse1 mechanism had gaps (missing support for storage, AMP, distributed, etc.), but **PyTorch 2.1** introduced many enhancements to make PrivateUse1 a first-class path for new device backends [3]. In short, PrivateUse1 acts as a placeholder "device type" that extension libraries can rename and fully utilize for custom hardware.

Tenstorrent's TT-Metal and PyTorch2.0_TTNN leverage this PrivateUse1 mechanism to integrate Tenstorrent's AI accelerators (like Wormhole, Blackhole chips) into PyTorch. **TT-NN** (the software stack in TT-Metal) is described as "a Python & C++ Neural Network op library" [4] that provides kernels and a programming model (TT-Metalium) for Tenstorrent devices. By using PrivateUse1, Tenstorrent can implement PyTorch tensor operations on their hardware without needing a bespoke dispatch key in upstream PyTorch. The companion repository **pytorch2.0_ttnn** is the PyTorch extension that ties into TT-Metal, enabling PyTorch to run models on Tenstorrent devices either in *eager mode* or via a *compiled graph mode* (using `torch.compile`) [5] [6].

Eager Mode Integration in TT-Metal (TTNN)

To support **eager execution** on a new device, several pieces of PyTorch's backend interface must be implemented. TT-Metal's `torch_ttnn` extension (in the `pytorch2.0_ttnn` repo) implements these components using the PrivateUse1 dispatch key:

- **Custom Kernel Implementations:** TT-NN provides high-performance implementations for many ATen operators on Tenstorrent hardware. These are registered to the PyTorch dispatcher under the PrivateUse1 key. For example, TTNN likely defines an `add` kernel (and others like conv, matmul, etc.) and registers it as: `TORCH_LIBRARY_IMPL(aten, PrivateUse1, m) { m.impl("add.Tensor", TORCH_FN(my_add_kernel)); … }` [7]. Any operation not explicitly implemented on the TT device will trigger a fallback. In TT-Metal's case, a **CPU fallback** is set up: they register a boxed fallback that uses `at::native::cpu_fallback()` to execute the op on CPU if the inputs are on the TT device [8]. This ensures functional correctness – unsupported ops run on CPU – albeit with a performance cost. (The TT Buda docs confirm that if certain ops aren't supported on Tenstorrent, the framework can route those sub-graphs to a CPUDevice automatically [9].)

```
// Pseudo-code illustrating custom add and fallback:
at::Tensor my_add_kernel(const at::Tensor& self, const at::Tensor& other,
at::Scalar alpha) {
    // ... implement add on Tenstorrent device ...
```

```
}
TORCH_LIBRARY_IMPL(aten, PrivateUse1, m) {
    m.impl("add.Tensor", TORCH_FN(my_add_kernel));  // register custom kernel
}
void custom_cpu_fallback(const c10::OperatorHandle& op, torch::jit::Stack*
stack) {
    // Fallback for ops not implemented on TT device:
    at::native::cpu_fallback(op, stack);
}
TORCH_LIBRARY_IMPL(_, PrivateUse1, m) {

m.fallback(torch::CppFunction::makeFromBoxedFunction<&custom_cpu_fallback>());
}
```

Listing: **Registering a custom op and CPU fallback for PrivateUse1** [7] [8]. In TT-Metal's TTNN, a similar pattern is used – all supported ops have TT-specific kernels, and a fallback ensures any unimplemented operation runs on CPU rather than erroring out.

- **Memory Allocation & Storage:** In order for tensors to actually reside on a new device, the backend must handle memory allocation. TT-Metal's device allocator likely interfaces with Tenstorrent drivers to allocate device memory. (If the device has unified memory or a driver API, the allocator wraps that. In a dummy setup, one could even allocate CPU memory but mark it as device memory.) In PyTorch, one typically creates a tensor on a custom device by constructing a `TensorImpl` with the `DispatchKeySet` including `PrivateUse1` (and its autograd key) and associating it with a Storage that uses the custom allocator [10] [11]. TTNN handles this internally when you create tensors on the TT device. For example, calling `torch.empty(..., device="tt:0")` (assuming `tt` is the alias for PrivateUse1) will invoke TTNN's custom `empty` kernel. That kernel would allocate memory on the Tenstorrent device (via TT-Metalium) and return a tensor with `device=PrivateUse1`. If such a kernel wasn't provided, `torch.empty` on a PrivateUse1 device would fail (since factory functions require an implementation to return a tensor on that device). Thus, TTNN implements basic factory ops (like empty, ones, zeros, etc.) for its device.

- **Device Guards and Streams:** PyTorch uses a `DeviceGuardImplInterface` for each device type to handle setting the current device, managing streams, and events. TTNN registers its DeviceGuard so that context managers like `torch.cuda.device(...)` or autograd's device save/restore work for Tenstorrent devices. For instance, TTNN would have: `C10_REGISTER_GUARD_IMPL(PrivateUse1, TtDeviceGuard);` during initialization [12]. This allows PyTorch to treat `PrivateUse1` similarly to CUDA (e.g., saving the current active TT device index, switching it as needed). In practice, TT's `torch_ttnn` likely uses the guard to set the active device in the underlying driver when PyTorch changes the current device.

- **Random Number Generator:** To support ops like `torch.rand(...)` on the TT device, a custom `GeneratorImpl` was likely implemented. PyTorch allows dynamic registration of a generator for PrivateUse1 [13]. TTNN would provide a `GeneratorImpl` that perhaps seeds and produces random numbers on the Tenstorrent hardware (or on host if necessary). The integration is done via `REGISTER_GENERATOR_PRIVATEUSE1(MyGeneratorCtor)` [14]. If not implemented, requesting

random tensors on the TT device would either fail or fall back to CPU. (Given Tenstorrent supports training, they likely implemented this to avoid unnecessary device transfers.)

- **Autograd Support:** For most ops, if you implement the forward on your device, PyTorch's autograd can use the CPU or default autograd formula as long as the backward *sub-ops* are available on some device. PyTorch will automatically route gradient computations through AutogradPrivateUse1 kernels if provided [15] [16], or use the default (often device-agnostic) ones if possible. TTNN appears to handle autograd mostly by leveraging PyTorch's existing formulas – their model support table shows high accuracy, implying gradients are correct. In cases where the default backward is not device-agnostic, a backend can override it. The tutorial shows an example of registering a custom autograd Function for an op [17] [18], but this is advanced and likely done only if needed. TTNN's initial focus was enabling the forward path on Tenstorrent; the backward path might fall back to CPU for certain ops if not implemented on device. (Full eager support ideally means both forward and backward kernels exist on the device or at least fallback safely.)

- **Automatic Mixed Precision (AMP):** If Tenstorrent hardware supports specific dtypes (like BF16, FP16), TTNN can integrate with PyTorch's AMP autocast. The PrivateUse1 integration supports registering autocast kernels and querying supported dtypes [19] [20]. TTNN likely registers a device module (`torch._register_device_module('ttnn', ...)`) that provides `get_amp_supported_dtype()` etc., so that `torch.autocast` works with the TT device. This is a detail that improves user experience by allowing mixed precision training on Tenstorrent accelerators.

- **Device Naming and Utilities:** By default, a tensor on PrivateUse1 shows `device='privateuse1:0'`. To make this user-friendly, PyTorch allows **renaming** the PrivateUse1 device type to a custom name [21] [22]. TTNN does this under the hood. For example, it might call `c10::register_privateuse1_backend("tt")` (or `"ttnn"`) so that users can refer to devices as `"tt:0"`. Indeed, in Tenstorrent's examples, you see `device = ttnn.open_device(0); model.to(torch_ttnn.ttnn_device_as_torch_device(device))` [23] – this implies that `torch_ttnn.ttnn_device_as_torch_device` returns a `torch.device` with type "ttnn" (alias for PrivateUse1) using the opened device's index/handle. After renaming, convenience methods can be generated: PyTorch offers `torch.utils.generate_methods_for_privateuse1_backend(...)` to add methods like `tensor.ttnn()` and properties `tensor.is_ttnn` automatically [24]. It's not clear if TTNN explicitly enabled those, but they did provide an API (`ttnn.from_torch(...)`, `ttnn.to_torch(...)`) for moving data between PyTorch (CPU) and TT device [25] [26]. In any case, once the device is renamed and opened, moving a model to the TT device is as simple as `.to("ttnn:0")` in principle. (The TTNN quick-start uses the helper function, but it accomplishes the same end result [27].)

Using TTNN in Eager Mode: With all the above in place, you can run PyTorch modules on Tenstorrent hardware much like on CUDA. The quick-start example shows two ways:

- *Option 1: Eager Mode* – Open a Tenstorrent device and move the model to it, then run forward passes normally [5] [28]. Behind the scenes, each operation on tensors now dispatches to TTNN's kernels. If an op is not supported on TT, the fallback will execute it on CPU (which could be slower, but ensures the model runs to completion). For example:

```
import torch, torch_ttnn, ttnn
model = YourModel()
device = ttnn.open_device(device_id=0)                        # initialize
Tenstorrent device (e.g., Wormhole card)
torch_dev = torch_ttnn.ttnn_device_as_torch_device(device)   # get a
torch.Device for PrivateUse1 ('tt:0')
model.to(torch_dev)  # move model parameters to Tenstorrent device
output = model(input_data)  # forward pass executes on Tenstorrent
```

Under the hood, `model.to(torch_dev)` will invoke TTNN's `to`-device logic (copying tensors to device memory). If any layer calls an op that TTNN doesn't implement, the registered fallback will copy those tensors to CPU, run the CPU kernel, and copy the result back to the TT device [8]. This allows even partially supported models to run (though frequent fallback indicates performance will suffer, as seen in their model table where some models had many "to/from device ops").

- *Option 2: Compile Mode* – The TTNN project also integrates with `torch.compile` to JIT compile a graph for the Tenstorrent device [6]. This involves tracing the model and lowering to their backend (possibly via torch-mlir or their compiler stack). While not the focus of this question, it's worth noting that the **recommended path for performance is compiling** (so that latency and unsupported ops can be optimized). Eager mode is most useful for debugging or quick iteration, but full performance comes from graph compilation.

In summary, **TT-Metal (TTNN)** extends PyTorch by providing a PrivateUse1 backend named "ttnn" with custom kernels, memory management, and integration points for autograd and device management. This enables *eager execution* on Tenstorrent hardware with minimal changes to user code (just move the model to `device='ttnn:0'`). The heavy lifting is done in C++ within TT-Metal (for kernels, allocator, etc.) and in the `torch_ttnn` Python/C++ glue code for registration.

Tutorial: Adding a New Device via PrivateUse1 (Step-by-Step)

To fully understand and **implement your own device backend** (similar to what TT-Metal does) on a local machine, here is a step-by-step guide. This "toy" example assumes you *don't have actual hardware* – we'll show how to create a fake device that simply uses CPU under the hood, which is useful for learning and testing. (Indeed, PyTorch's own test extension for PrivateUse1 demonstrates a custom device that "secretly" stores data in CPU memory [29].)

Prerequisites: You should have PyTorch installed (version 2.1 or later for best PrivateUse1 support) and a C++14/C++17 compiler. We'll use PyTorch's C++ Extension API to build our custom backend as a Python module.

1. Define a Custom Backend Module (C++): Create a C++ source file (e.g. `my_backend.cpp`) that will register our device. In it, include the PyTorch C++ APIs:

```
#include <torch/extension.h>
#include <ATen/ATen.h>
```

```
#include <c10/core/DeviceType.h>
#include <c10/core/Allocator.h>
#include <c10/core/DeviceGuard.h>
```

We first **register our backend name**. PyTorch allows doing this in C++ by calling `c10::register_privateuse1_backend("mydev")`. This gives the device a user-friendly name:

```
TORCH_LIBRARY_FRAGMENT(mydev, m) {  // Library fragment for our backend
(optional)
    c10::register_privateuse1_backend("mydev");
}
```

Now the device type `"mydev"` will map to `PrivateUse1`. (Alternatively, you can call `torch.rename_privateuse1_backend("mydev")` in Python after loading the extension [22] – either approach works.)

2. Implement Device-Specific Ops: For our toy example, implement a minimal set of operations: - A factory method to create tensors on the device (e.g. an `empty` or `ones` function). - A simple arithmetic op (e.g. add) to demonstrate kernel dispatch. - A fallback mechanism for anything else.

Memory Allocation: Since we have no real device, we'll allocate CPU memory but mark it as our device. We can use the default CPU allocator and then wrap the pointer in a tensor with PrivateUse1 dispatch key. For example:

```
at::Tensor mydev_empty(c10::IntArrayRef sizes, c10::ScalarType dtype,
c10::optional<at::MemoryFormat> optional_memory_format) {
    // Allocate CPU memory for given size:
    size_t nbytes = at::elementSize(dtype) * c10::multiply_integers(sizes);
    auto cpuAlloc = c10::GetCPUAllocator();
    void* cpuPtr = cpuAlloc->raw_allocate(nbytes);
    // Create Storage and TensorImpl for PrivateUse1:
    c10::DataPtr dptr(cpuPtr, cpuPtr, &(c10::CPUAllocator::deallocate),
c10::Device(c10::DeviceType::CPU));
    auto storageImpl = c10::make_intrusive<at::StorageImpl>(
         at::StorageImpl::use_byte_size_t(),
         nbytes,
         std::move(dptr),
         cpuAlloc,
         /*resizable=*/true);
    // Construct a TensorImpl with PrivateUse1 dispatch key:
    c10::DispatchKeySet keys({c10::DispatchKey::PrivateUse1,
c10::DispatchKey::AutogradPrivateUse1});
    auto tensorImpl =
c10::make_intrusive<at::TensorImpl>(std::move(storageImpl), keys, dtype);
    tensorImpl->set_sizes_contiguous(sizes);  // set tensor shape
```

```
    return at::Tensor(std::move(tensorImpl));
}
```

This is a bit low-level, but essentially we allocate memory and manually create a Tensor. In a real backend, you'd use your device's allocator/driver API here. (PyTorch's tutorial notes that for storage-less backends you'd use `OpaqueTensorImpl`, but for simplicity we use a storage with CPU memory [10].)

Custom Add: We implement an `add` that operates on our device. Since our data is actually in CPU memory, we can directly perform the addition on the CPU pointers:

```
at::Tensor mydev_add(const at::Tensor& a, const at::Tensor& b, const
at::Scalar& alpha) {
    TORCH_CHECK(a.device().type() == c10::DeviceType::PrivateUse1, "Tensor 'a'
not on mydev");
    TORCH_CHECK(b.device().type() == c10::DeviceType::PrivateUse1, "Tensor 'b'
not on mydev");
    // Ensure dtype match and contiguous:
    at::Tensor result = mydev_empty(a.sizes(), a.scalar_type(), c10::nullopt);
    float* ap = (float*)a.data_ptr();   // (Assuming float for simplicity)
    float* bp = (float*)b.data_ptr();
    float* rp = (float*)result.data_ptr();
    size_t n = a.numel();
    float alpha_val = alpha.to<float>();
    for (size_t i = 0; i < n; ++i) {
        rp[i] = ap[i] + alpha_val * bp[i];
    }
    return result;
}
```

This function adds two same-shaped float tensors. It uses raw pointers to CPU data. In a real scenario, you'd call device-specific vector operations or offload to your hardware.

3. Register Kernels with the Dispatcher: Using the `TORCH_LIBRARY_IMPL` macro, we tie these C++ functions to PyTorch's operator names for our backend:

```
TORCH_LIBRARY_IMPL(aten, PrivateUse1, m) {
    m.impl("empty.memory_format",         TORCH_FN(mydev_empty));  // override
aten::empty for mydev
    m.impl("add.Tensor",                 TORCH_FN(mydev_add));    // override
aten::add.Tensor
}
```

Now, whenever PyTorch's dispatcher needs to execute `aten::empty` or `aten::add.Tensor` on a `mydev` tensor, it will call our functions. We also should register a fallback for any other ops:

```
// Fallback: unimplemented ops go to CPU
static void mydev_fallback(const c10::OperatorHandle& op, torch::jit::Stack*
stack) {
    at::native::cpu_fallback(op, stack);
}
TORCH_LIBRARY_IMPL(_, PrivateUse1, m) {
    m.fallback(torch::CppFunction::makeFromBoxedFunction<&mydev_fallback>());
}
```

This fallback uses PyTorch's built-in CPU fallback utility [8]. It will internally copy any `mydev` tensors to CPU, run the op on CPU, and (if possible) copy results back to mydev. (For example, if you call `y = x * 2` on a `mydev` tensor without a registered kernel, this fallback will move `x` to CPU, do the multiplication, then move `y` back to mydev.)

4. DeviceGuard Implementation (optional): For a full integration, implement a struct inheriting `c10::DeviceGuardImplInterface` to handle `setDevice`, `getDevice`, `exchangeDevice`, etc., for your device. In our dummy case, since the "device" *is* CPU memory, we might not need a complex guard (the default might suffice). But to follow best practices:

```
struct MyDevGuardImpl : public c10::impl::DeviceGuardImplInterface {
    c10::DeviceType type() const override { return
c10::DeviceType::PrivateUse1; }
    c10::Device exchangeDevice(c10::Device newDevice) const override {
        // (No real device state to switch in this toy example)
        int newIndex = newDevice.index();
        int oldIndex = 0;
        currentDevice_ = newIndex;
        return c10::Device(c10::DeviceType::PrivateUse1, oldIndex);
    }
    c10::Device getDevice() const override {
        return c10::Device(c10::DeviceType::PrivateUse1, currentDevice_);
    }
    void setDevice(c10::Device device) const override { currentDevice_ =
device.index(); }
    void uncheckedSetDevice(c10::Device device) const noexcept override {
currentDevice_ = device.index(); }
    // ... (stream and event related methods would go here if needed)
    static thread_local int currentDevice_;
};
thread_local int MyDevGuardImpl::currentDevice_ = 0;
C10_REGISTER_GUARD_IMPL(PrivateUse1, MyDevGuardImpl);
```

This simplistic guard just tracks a "current device index" (always 0 in our case). In a real backend, this would interact with the hardware context or driver (e.g., set the active GPU or accelerator).

5. Register a Custom Generator (optional): If you want `torch.randn(..., device="mydev")` to work, define a `c10::GeneratorImpl` for `PrivateUse1`. For brevity, we'll skip detailed implementation, but it involves subclassing `c10::GeneratorImpl` and implementing methods like `random()` and `clone()`. You then register it with `REGISTER_GENERATOR_PRIVATEUSE1(MyGeneratorImplCtor)` [30]. Without this, requesting random numbers on the device will likely fall back to CPU or error out.

6. Build and Load the Extension: You can compile this backend as a Python extension. Using PyTorch's `torch.utils.cpp_extension`, for example:

```
from torch.utils.cpp_extension import load
extra_include_paths = [torch.utils.cpp_extension.include_paths()]  # ensure
PyTorch headers
mydev_ext = load(name="mydev_ext", sources=["my_backend.cpp"], extra_cflags=['-
std=c++17'])
```

This will compile and load the C++ code, making the symbols (like `mydev_empty`, `mydev_add`) available to Python through the dispatcher. Ensure your compiled extension is loaded **before** using the device.

7. Test the New Device in Python: After loading the extension, do the following in Python:

- **Rename PrivateUse1** to your backend name if not done in C++: `torch.rename_privateuse1_backend("mydev")` [31]. (If we called `register_privateuse1_backend` in C++, this is already done.)
- **Generate convenience methods** (optional): `torch.utils.generate_methods_for_privateuse1_backend(for_tensor=True, for_storage=True)`. This will enable calls like `tensor.mydev()` to copy to device, and `tensor.is_mydev` to check device type [24].
- **Allocate a tensor on the new device:** You can now do:

  ```
  x = torch.ones((3,3), device="mydev:0")
  y = torch.ones((3,3), device="mydev:0")
  z = x.add(y)  # uses mydev_add kernel
  print(z)      # should be on mydev:0
  ```

  Behind the scenes, `torch.ones` will call our `empty` kernel (to allocate) then a fill kernel. We didn't implement fill, but because we set a fallback, the fill operation (`aten::fill_`) will fall back to CPU: PyTorch will temporarily copy the tensor to CPU, fill it, and copy back. Similarly, `x.add(y)` matches our `add.Tensor` registration, so `mydev_add` is called to compute the result on our "device". The result `z` is a tensor on `mydev:0` as well. If you try an operation we didn't implement (say `x.sum()`), the fallback will trigger – PyTorch will copy `x` to CPU, compute the sum, and since the sum is a CPU number (scalar), it will just return a CPU tensor or Python number. The mechanism is transparent to the user, aside from potential performance differences.

- **Verify data movement:** In our dummy backend, we can check that the data actually lives in CPU memory by comparing results. For example, `print(z.cpu())` should show the same values as `z` (since in reality they are already in CPU memory). In a real backend, of course, `.cpu()` would trigger an actual device-to-host transfer.

8. Extend as Needed: With this skeleton, you can add more operators or features: - Implement more kernels (matrix multiply, convolutions) using your hardware API. - Integrate streams/events if your device supports async execution. - Hook into distributed (Processes can register collective ops for PrivateUse1, though as of 2.1 this might be future work [32]). - Write proper backward kernels or rely on autograd fallback (which may send grads to CPU if forwards were on CPU fallback).

9. No Physical Hardware Required (for development): As demonstrated, you can develop a new backend without actual hardware by using CPU under the hood. This is useful for understanding the integration points. PyTorch's own test plugin essentially does exactly that – it pretends to be a new device but uses CPU memory and compute [29]. This way, you can iteratively build out support in a controlled environment. Once the scaffolding works (allocations, basic ops, etc.), you would replace the dummy implementations with real interactions to your accelerator's SDK/driver. This strategy answers the question: yes, you *can* develop and test a custom device locally. The key is that the PrivateUse1 mechanism abstracts the device type, so as long as your code produces correct tensors and results, PyTorch doesn't differentiate whether it's a real device or a simulated one.

10. Summary of Process: Integrating a new device with PyTorch's eager mode involves: - **Registering device kernels and fallback** for core ops so that computations on the device either execute natively or safely fall back [33] [8]. - **Providing backend-specific infrastructure** like memory allocator, device guard, and generator so that PyTorch can manage your device's resources (memory, streams, random state) just as it does for CPU/CUDA. - **Naming and user API** enhancements to make the experience seamless (device strings, `.to()` support, etc. – much of which is handled by the `rename_privateuse1_backend` and method generation utilities [22] [24]). - **Testing with toy examples:** e.g., create tensors on the device, perform arithmetic, move data to/from CPU, and ensure correctness.

By following these steps, you can achieve a *full eager-mode integration* of a custom device. Tenstorrent's TT-Metal did all of the above for their hardware – implementing dozens of kernels optimized for their chips, setting up the `torch_ttnn` extension to register those, and providing a smooth Python interface. With this understanding, you should be able to read the TT-Metal code with clarity: recognize where they register ops, how they use `PrivateUse1` to tie into PyTorch, and what additional features they added (like the compile path) on top of the basic eager support.

References:

- PyTorch Tutorial – *Facilitating New Backend Integration by PrivateUse1* [1] [3] [33] [8] (explains the motivation and new support in 2.1, plus code snippets for kernel and fallback registration).
- PyTorch Tutorial – *Extending dispatcher for a new backend in C++* [10] [17] [18] (in-depth guide on implementing a backend, including autograd overrides).
- Tenstorrent TTNN Quick Start (README) – demonstrates usage of the TT backend in eager and compiled modes [5] [28].
- PyTorch Dev Discussion – *Custom device example* – points to the open registration test which uses CPU as a backing store [29], a helpful minimal example.

1 2 3 7 8 12 13 14 15 16 19 20 21 22 24 30 31 32 33 Facilitating New Backend Integration by PrivateUse1 — PyTorch Tutorials 2.8.0+cu128 documentation
https://docs.pytorch.org/tutorials/advanced/privateuseone.html

4 raw.githubusercontent.com
https://raw.githubusercontent.com/tenstorrent/tt-metal/main/README.md

5 6 23 27 28 GitHub - tenstorrent/pytorch2.0_ttnn: ⭐ TTNN Compiler for PyTorch 2 ⭐ Enables running PyTorch models on Tenstorrent hardware using eager or compile path
https://github.com/tenstorrent/pytorch2.0_ttnn

9 User Guide — TT Buda documentation
https://docs.tenstorrent.com/pybuda/latest/user_guide.html

10 11 17 18 Extending dispatcher for a new backend in C++ — PyTorch Tutorials 2.8.0+cu128 documentation
https://docs.pytorch.org/tutorials/advanced/extend_dispatcher.html

25 26 Converting PyTorch Model to TT-NN — TT-NN documentation
https://docs.tenstorrent.com/tt-metal/latest/ttnn/ttnn/converting_torch_model_to_ttnn.html

29 Any simple example about the new way to register custom device through PrivateUse1 - frontend API - PyTorch Developer Mailing List
https://dev-discuss.pytorch.org/t/any-simple-example-about-the-new-way-to-register-custom-device-through-privateuse1/1561